

BÁO CÁO TỔNG KẾT ĐỒ ÁN MÔN HỌC

Lớp: IT003.Q21.TTNT

SINH VIÊN THỰC HIỆN:

Mã sinh viên: 25521290 **Họ và tên:** Nguyễn Thành Thiện Nhân

TÊN ĐỀ TÀI:

Xây dựng Game Thủ Thành (Tower Defense) ứng dụng các Thuật toán Tìm đường và Cấu trúc dữ liệu nâng cao.

CÁC NỘI DUNG CẦN BÁO CÁO

1. Giới thiệu đồ án

a. Mô tả chung về ứng dụng

Đồ án là một tựa game chiến thuật Thủ Thành (Tower Defense) trên nền đồ họa 2D, được phát triển bằng ngôn ngữ Python kết hợp thư viện Pygame. Nhiệm vụ của người chơi là sử dụng tài nguyên để xây dựng các tháp phòng thủ (Tower) trên một bản đồ dạng lưới nhằm ngăn chặn các đợt tấn công (Wave) của quái vật di chuyển về phía căn cứ (Base). Điểm đặc biệt của game là việc tích hợp các Cấu trúc dữ liệu và Thuật toán cốt lõi để quản lý logic game và tạo AI tìm đường cho quái vật.

b. Các CTDL và giải thuật được sử dụng (What, Why)

Linked List (Danh sách liên kết đôi/đơn):

- Sử dụng để lưu trữ và quản lý danh sách quái vật đang tồn tại trên bản đồ.
- Trong game Tower Defense, quái vật liên tục được sinh ra và bị tiêu diệt (xóa khỏi bộ nhớ). Linked List cho phép thao tác chèn (insert) và xóa (delete) phần tử với độ phức tạp $O(1)$ tại vị trí con trỏ, giúp quản lý bộ nhớ linh hoạt và tối ưu hiệu suất hơn so với dùng mảng (Array).

Queue (Hàng đợi):

- Dùng để quản lý hệ thống sinh quái theo từng đợt (Wave system).
- Đảm bảo nguyên tắc FIFO (First In First Out), quái vật nào được đưa vào hàng đợi trước sẽ được sinh ra trên bản đồ trước theo đúng thứ tự thiết kế của màn chơi.

Priority Queue (Hàng đợi ưu tiên):

- Cài đặt AI cho Tháp phòng thủ để chọn mục tiêu tấn công.
- Tháp cần đưa ra quyết định bắn dựa trên độ ưu tiên (ví dụ: ưu tiên bắn quái vật đang gần căn cứ nhất hoặc quái vật có lượng máu thấp nhất). Priority Queue giúp lấy ra (pop) mục tiêu có độ ưu tiên cao nhất một cách nhanh chóng.

Thuật toán BFS (Breadth-First Search):

- Thuật toán tìm kiếm theo chiều rộng dùng để tạo AI tìm đường trên bản đồ lưới 2D.
- Môi trường game là lưới không trọng số. BFS đảm bảo luôn tìm được đường đi ngắn nhất từ điểm sinh quái đến căn cứ. Đồng thời, thuật toán dễ dàng tính toán lại lộ trình khi người chơi đặt thêm tháp (tạo thành chương ngại vật).

2. Quá trình thực hiện

a. Tuần 1

Trong tuần 1, tiến độ công việc tập trung vào khâu phân tích yêu cầu, chốt ý tưởng thiết kế và khởi tạo môi trường:

- Hoàn thiện tài liệu thiết kế (Game Design Document) cơ bản, xác định rõ các yêu cầu về Cấu trúc dữ liệu và Thuật toán cần sử dụng theo chuẩn môn học.
- Cài đặt môi trường phát triển: Cài đặt Python, cài đặt thư viện đồ họa Pygame.
- Khởi tạo project repository trên hệ thống Git (để public) phục vụ cho việc quản lý mã nguồn và version control.
- Phân tích thiết kế các class cơ bản sẽ tự triển khai (code from scratch): Node, LinkedList, Queue.

b. Tuần 2

Trong tuần 2, tiến độ công việc bước vào giai đoạn Coding (triển khai mã nguồn), tập trung giải quyết các bài toán về Cấu trúc dữ liệu và xây dựng khung giao diện:

- **Tự lập trình Cấu trúc dữ liệu (From scratch):** Xây dựng file data_structures.py. Tự triển khai các lớp Node, LinkedList (quản lý danh

sách quái vật) và Queue (quản lý Wave system) mà không sử dụng các thư viện có sẵn của Python.

- **Viết Docstring và phân tích $O(n)$:** Hoàn thiện docstring theo chuẩn PEP-8 cho tất cả các phương thức (thêm, xóa, tìm kiếm). Phân tích chi tiết Độ phức tạp thời gian (Time Complexity) và Không gian (Space Complexity).
- **Xây dựng khung giao diện (UI) bằng Pygame:**
 - Khởi tạo cửa sổ game, vẽ hệ thống bản đồ dạng lưới (Grid 2D).
 - Xác định và hiển thị các vị trí cốt lõi: Điểm sinh quái (Spawn Point) và Căn cứ (Base).
- **Thử nghiệm đa nền tảng (Bổ sung):** Ngoài việc chạy ổn định trên Windows 11, em đã thử nghiệm đóng gói mã nguồn thành file APK và chạy test giao diện thành công trên thiết bị Android (Xiaomi Pad 8 Pro). Đây là bước kiểm thử tương thích phụ, không làm thay đổi trọng tâm thuật toán của đồ án.
- **Quản lý phiên bản:** Đẩy các thay đổi mã nguồn (commit) lên kho lưu trữ Github.

c. Tuần 3

Trong tuần 3, em tập trung vào việc áp dụng thuật toán BFS vào game và xử lý thao tác nhấp chuột:

- **Cài đặt Thuật toán BFS:** Viết hàm tìm đường bằng thuật toán BFS. Thuật toán này sẽ duyệt qua ma trận lưới 2D để tìm đường đi ngắn nhất cho quái vật từ điểm xuất phát đến căn cứ.
- **Tự viết Priority Queue (Hàng đợi ưu tiên):** Tự code thêm class Priority Queue (không dùng thư viện ngoài) để Tháp lưu danh sách quái vật trong tầm bắn, từ đó ưu tiên lấy ra con quái vật đang gần căn cứ nhất để bắn.
- **Tương tác chuột & Cập nhật đường đi:** Cài đặt sự kiện click chuột trong Pygame để người chơi đặt Tháp. Mỗi khi đặt một Tháp xuống (coi như một vật cản mới), chương trình sẽ tự động gọi lại hàm BFS để tính đường đi mới giúp quái vật biết cách né Tháp.

- **Bổ sung Docstring:** Ghi chú code đầy đủ và phân tích độ phức tạp thời gian cho hàm BFS ($O(V+E)$) theo đúng yêu cầu môn học.
- **Quản lý phiên bản:** Lưu mã nguồn và đẩy lên Github.

d. Tuần 4 (Hoàn thiện & Tối ưu hóa)

Trong tuần cuối cùng, em tập trung vào việc hoàn thiện logic game, cân bằng thông số và đóng gói dự án:

- **Tối ưu hóa Thuật toán & CTDL:** Tối ưu hóa lại cấu trúc Linked List và Priority Queue để giảm thiểu tình trạng giật lag (drop FPS) khi số lượng quái vật sinh ra từ Queue quá đông.
- **Hoàn thiện Gameplay Loop:** Cài đặt các cơ chế thắng/thua (Win/Lose condition). Nếu quái vật đến được căn cứ (độ dài đường đi BFS = 0), máu của căn cứ sẽ giảm.
- **Hoàn thiện Đồ họa (UI/UX):** Thay thế các ô vuông màu sắc bằng hình ảnh (Sprite) cơ bản cho Tháp, Quái vật, và Căn cứ để giao diện trực quan hơn.
- **Tổng kết tài liệu:** Rà soát lại toàn bộ docstring của các hàm, đảm bảo ghi rõ độ phức tạp ($O(n)$, $O(V+E)$) theo đúng chuẩn. Viết file README.md hướng dẫn cài đặt và chạy game trên Github.

3. Kết quả đạt được (Tổng kết dự án)

- **Về mặt kỹ thuật:** Đã tự lập trình thành công 100% các cấu trúc dữ liệu và giải thuật cốt lõi (Linked List, Queue, Priority Queue, BFS) bằng Python thuần mà không dùng thư viện ngoài, đáp ứng đúng yêu cầu của môn học.
- **Về mặt ứng dụng:** Xây dựng hoàn chỉnh một tựa game Tower Defense có thể chơi được trên Pygame, quái vật có khả năng tự tìm đường thông minh và né vật cản theo thời gian thực.
- Đã xuất bản mã nguồn mở trên nền tảng Github kèm tài liệu hướng dẫn cài đặt.
- **Mức độ hoàn thành dự án:** 100%.

5. Phụ lục : Docstring

(Ví dụ trích xuất docstring của hàm BFS)

python

```
def bfs_find_path(grid, start, target):
```

```
    """
```

Thuật toán BFS tìm đường đi ngắn nhất trên ma trận 2D.

Tham số:

grid (list): Ma trận lưới 2D (0 là đường đi được, 1 là Tháp/vật cản).

start (tuple): Tọa độ ô bắt đầu.

target (tuple): Tọa độ ô đích (Căn cứ).

Trả về:

list: Danh sách các tọa độ tạo thành đường đi, nếu không có đường thì trả về mảng rỗng.

Độ phức tạp:

- Thời gian (Time Complexity): $O(V + E)$ với V là số ô trên lưới và E là số cạnh nối giữa các ô.

- Không gian (Space Complexity): $O(V)$ vì cần dùng Queue và mảng Visited để lưu các ô đã duyệt.

```
    """
```

Pass

Link GITHUB: <https://github.com/bersious/Shapes-Wars>